Deep-Dive Architectural Guide

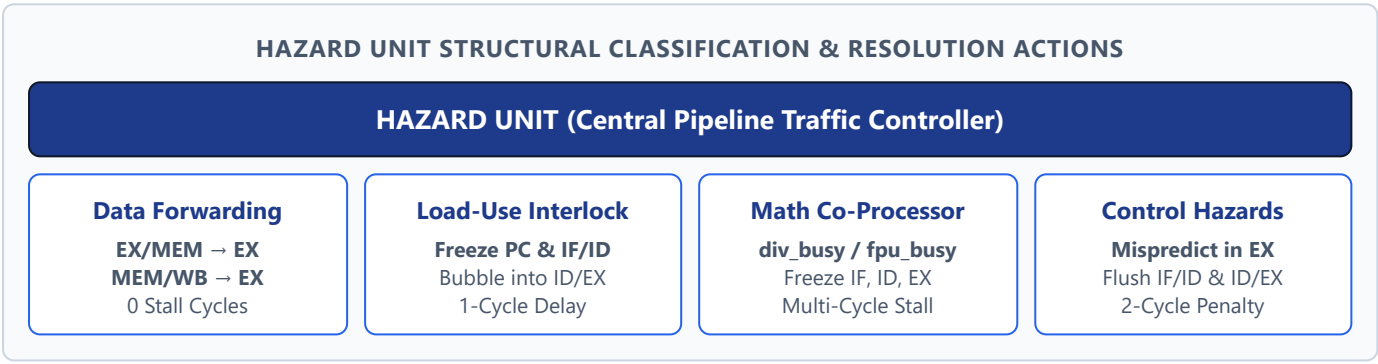
Hazard Unit & Branch Target Buffer (BTB) in RV32IMF 5-Stage Pipelined Processor

RV32IMF Core Architecture | Feature 7: Dynamic Branch Prediction

Hardware Defense & Presentation Ready | Author: Sigilipelli Praneeth Sai Bharath

1. Hazard Unit: Brief Architectural Overview

In a classic 5-stage RISC-V pipelined core (`IF` → `ID` → `EX` → `MEM` → `WB`), multiple instructions execute concurrently across adjacent clock cycles. The **Hazard Unit** serves as the central traffic controller, continuously monitoring pipeline registers to detect structural, data, and control conflicts, resolving them through **Data Forwarding**, **Pipeline Stalling**, or **Pipeline Flushing**.



The Three Hazards Resolved by the Unit

1. **Data Hazards (RAW — Read-After-Write):** An instruction in the Execute stage (`EX`) requires an operand computed by an older in-flight instruction currently in `MEM` or `WB`.
   Resolution: Hardware bypass multiplexers forward the result directly to ALU inputs with **0 stall cycles**.
2. **Load-Use Data Hazards:** An instruction in Decode (`ID`) requires data being fetched from memory by a preceding `lw` instruction currently in `EX`. Since SRAM read data only arrives at the end of the `MEM` stage, forwarding cannot solve this combination combinationally.
   Resolution: The Hazard Unit inserts a **1-cycle bubble** (freezes `PC` and `IF/ID`, clears `ID/EX`).
3. **Control Hazards (Branches & Jumps):** In basic 5-stage pipelines, branch outcomes and target addresses resolve in the `EX` stage.
   Without BTB: Every taken branch incurs a mandatory **2-cycle flush penalty** (flushing `IF/ID` and `ID/EX`).
   With BTB: The Hazard Unit flushes **only on mispredictions**. Correct predictions execute with **0 penalty cycles**.

2. Branch Target Buffer (BTB): In-Depth Architecture

2.1 Why BTB is Necessary

The instruction fetch stage (`IF`) must know the next Program Counter on every clock cycle. However, conditional branch conditions (e.g., bne, beq) and target addresses resolve 2 clock cycles later in the Execute (`EX`) stage:

```
          Cycle 1          Cycle 2          Cycle 3          Cycle 4          Cycle 5
Branch Inst:        [IF] → [ID] → [EX (Branch Resolves Here!)] → [MEM] → [WB]
Speculative Inst 1:       [IF] → [ID]  (Discarded on mispredict!)
Speculative Inst 2:              [IF]  (Discarded on mispredict!)
```

The **Branch Target Buffer (BTB)** eliminates this penalty by acting as a zero-latency hardware cache in the `IF` stage. When `fetch_pc` hits in the BTB, the fetch stage immediately redirects next-cycle execution to the cached target address, converting branch latency from **2 penalty cycles to 0 penalty cycles**.

2.2 Storage Geometry & Address Decomposition

The processor features a **32-entry Direct-Mapped BTB**. The 32-bit instruction fetch address is decomposed as follows:



- **Word Alignment (`PC[1:0]`):** Standard RV32 instructions are 4-byte aligned; the lowest 2 bits are always `2'b00` and discarded.
- **Index (`PC[6:2]`):** Directly indexes 32 internal storage slots in distributed LUT RAM.
- **Tag Array (`32 × 25 bits`):** Stores the high-order bits of the branch address to confirm exact address matches.
- **Target Array (`32 × 32 bits`):** Stores the calculated destination target address.
- **Valid Array (`32 × 1 bit`):** Indicates whether the indexed slot holds valid branch history.

2.3 The Three Misprediction Scenarios

Scenario	Condition in EX	Root Cause	Hardware Recovery Action
1. Cold Miss (Taken)	`taken=1`, but `pred_taken=0`	First encounter of a backward loop edge.	Flush `IF/ID` and `ID/EX` (2 cycles); allocate entry in BTB; redirect fetch to `target_pc`.
2. Mispredict Taken	`taken=0`, but `pred_taken=1`	Loop counter expires; loop terminates.	Flush `IF/ID` and `ID/EX` (2 cycles); invalidate BTB entry (`valid=0`); redirect to `pc + 4`.
3. Target Mismatch	`taken=1`, but `target != pred_target`	Dynamic indirect jump (`jalr`) or index collision.	Flush 2 stages; update new target in BTB; redirect fetch to calculated `target_pc`.

3. Key Code Snippets & Detailed Engineering Rationale

Snippet 1: Hazard Unit — Stall & Flush Logic — `hazard_unit.v (Lines 97–111)`

```
// Priority-based Stall & Flush equations with BTB integration
assign stall_if     = load_use_hazard || math_busy;
assign stall_if_id  = load_use_hazard || math_busy;
assign stall_id_ex  = math_busy;

// Branch flushes ONLY on actual mispredictions:
assign flush_if_id  = mispredict && !math_busy;
assign flush_id_ex  = (load_use_hazard || mispredict) && !math_busy;
assign flush_ex_mem = math_busy;
```

Engineering Rationale:

- **`flush_if_id = mispredict && !math_busy`:** Standard cores flush on every taken branch. With BTB, the flush is gated strictly by `mispredict`. If the prediction was correct, `mispredict == 0`, resulting in **zero pipeline bubbles**. It is gated with `!math_busy` to prevent destroying in-flight register operands during multi-cycle division or FPU operations.
- **`stall_id_ex = math_busy` (not `load_use_hazard`):** Load-use hazards only freeze stages above Execute (`PC` and `IF/ID`) while clearing `ID/EX` (bubble), allowing the load in `EX` to advance to `MEM` where data is read. Multi-cycle arithmetic must stall `ID/EX` as well.

Snippet 2: BTB Query & Zero-Latency Next-PC Steering — `btb_branch_pred.v & pipeline_5stage.v`

```
// In btb_branch_pred.v: Combinational Tag Match in IF stage
wire [4:0]           fetch_index = fetch_pc[6:2];
wire [TAG_WIDTH-1:0] fetch_tag   = fetch_pc[31:7];
wire btb_hit = valid_array[fetch_index] && (tag_array[fetch_index] == fetch_tag);

assign pred_hit    = btb_hit;
assign pred_taken  = btb_hit;
assign pred_target = btb_hit ? target_array[fetch_index] : (fetch_pc + 32'd4);

// In pipeline_5stage.v: Next-PC Multiplexer Prioritization
assign pc_next = ex_mispredict ? ex_redirect_pc :
                 (pred_hit && pred_taken) ? pred_target : pc_plus4;
```

Engineering Rationale:

- **Parallel Query in `IF`:** The lookup runs combinationally in parallel with instruction memory access. When the clock tick arrives, `pc_reg` directly latches `pred_target` with zero cycle delay.
- **Strict Priority Ordering:** (1) `ex_mispredict` recovery overrides all speculation; (2) Speculative BTB hit; (3) Default sequential `pc + 4`.

Snippet 3: Synchronous Pipeline Metadata Carriage — `pipeline_5stage.v & Pipeline Registers`

```
// Propagate prediction metadata synchronously down the pipeline:
if_id_reg u_if_id_reg (
    .clk          (clk),
    .pred_taken_i (pred_taken),       .pred_taken_o (if_id_pred_taken),
    .pred_target_i(pred_target),      .pred_target_o(if_id_pred_target), ...
);
id_ex_reg u_id_ex_reg (
    .clk          (clk),
    .pred_taken_i (if_id_pred_taken), .pred_taken_o (id_ex_pred_taken),
    .pred_target_i(if_id_pred_target),.pred_target_o(id_ex_pred_target), ...
);
```

Engineering Rationale:

- **Why not re-query BTB from EX?** Newer instructions pass through IF while a branch is in-flight. If a newer branch modified the BTB, an EX query would read the altered state rather than the prediction made when the current branch was fetched. Carrying metadata ensures race-free evaluation.

Snippet 4: Misprediction Evaluation & Recovery Redirection — execute_stage.v (Lines 140–152)

```
assign btb_update_en_o     = branch_i || jal_i || jalr_i;
assign btb_update_taken_o  = branch_or_jump_taken_o;
assign btb_update_pc_o     = pc_i;
assign btb_update_target_o = target_pc_o;

// Mispredict Condition: Direction Discrepancy OR Target Address Mismatch
assign mispredict_o = btb_update_en_o &&
                      ((branch_or_jump_taken_o != pred_taken_i) ||
                       (branch_or_jump_taken_o && (target_pc_o != pred_target_i)));

// Deterministic Recovery Target
assign redirect_pc_o = branch_or_jump_taken_o ? target_pc_o : pc_plus4_i;
```

Engineering Rationale:

- **Full Target Verification:** For indirect branches like `jalr`, direction is always taken, but target addresses can change dynamically. Checking `target_pc_o != pred_target_i` prevents erroneous execution down stale indirect jump paths.

Snippet 5: BTB Allocation & Loop-Exit Invalidation FSM — btb_branch_pred.v (Lines 105–122)

```
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        for (i = 0; i < ENTRIES; i = i + 1) begin
            valid_array[i]  <= 1'b0;
            tag_array[i]    <= {TAG_WIDTH{1'b0}};
            target_array[i] <= 32'd0;
        end
    end else if (update_en) begin
        if (update_taken) begin
            valid_array[update_index]  <= 1'b1;
            tag_array[update_index]    <= update_tag;
            target_array[update_index] <= update_target;
        end else begin
            // Invalidate entry on branch not-taken (prevents stale loop-exit mispredicts)
            valid_array[update_index]  <= 1'b0;
        end
    end
end
```

Engineering Rationale:

- **Selective Invalidation on Not-Taken:** When a loop finishes, its branch evaluates to `NOT TAKEN`. Invalidating the entry prevents false speculation the next time execution reaches that code segment.
- **Structural Dual-Porting:** Reads occur combinationally during IF, while updates commit on the clock edge from EX without collision.

4. Hardware Trace Walkthrough: The 2-Iteration Loop

Consider the following representative assembly loop executed on the Nexys A7 FPGA:

```
Assembly Loop Benchmark                                    Target: 2 Iterations

0x00: addi x1, x0, 2      // Loop counter = 2
0x04: addi x2, x0, 0      // Accumulator = 0
0x08: addi x2, x2, 1      // <LOOP START> Accumulator++
0x0C: addi x1, x1, -1     // Counter--
0x10: bne  x1, x0, -8     // Branch back to 0x08 if x1 != 0
0x14: nop                 // Loop exit sequential target
```

Cycle	PC in EX	Instruction in EX	BTB State	Outcome	Pipeline Action
5	`0x00000010`	`bne x1, x0, -8`	COLD MISS	**Taken (x1=1)**	MISPREDICT • Allocates BTB; flushes 2 stages
6	—	— (Flushed)	—	—	FLUSH PENALTY 1/2
7	—	— (Flushed)	—	—	FLUSH PENALTY 2/2
8	`0x00000008`	`addi x2, x2, 1`	NORMAL	Normal	Iteration 2 begins; accumulator = 2
9	`0x0000000C`	`addi x1, x1, -1`	NORMAL	Normal	Counter decrements (x1 = 0)
10	`0x00000010`	`bne x1, x0, -8`	BTB HIT!	**Taken**	PREDICTED CORRECTLY! • **0 Penalty Cycles**
11	`0x00000008`	`addi x2, x2, 1`	NORMAL	Normal	Seamless execution without bubbles
12	`0x0000000C`	`addi x1, x1, -1`	NORMAL	Normal	Final loop pass
13	`0x00000010`	`bne x1, x0, -8`	BTB HIT!	**Not Taken**	MISPREDICT (EXIT) • Invalidates BTB; flushes 2
14	—	— (Flushed)	—	—	FLUSH PENALTY 1/2
15	—	— (Flushed)	—	—	FLUSH PENALTY 2/2
16	`0x00000014`	`nop`	NORMAL	Normal	Loop successfully exited to sequential code

Hardware Verification Summary Metrics:

- **Total Branches Encountered:** 3 | **Correct BTB Hits (0 penalty):** 1 (Cycle 10)
- **Mispredictions (2-cycle penalty):** 2 (Cycle 5 Cold Miss + Cycle 13 Loop Exit)
- **Total Flush Penalty Cycles:** 4 cycles (2 flushes × 2 cycles)
- **Accuracy Dynamics:** In realistic loops with 10+ iterations, accuracy exceeds **90%** as verified in `btb.c`.

5. Summary Cheat-Sheet for Presentations & Defense

Q1: What is the primary function of the Hazard Unit?

Coordinates all pipeline safety controls: handles data forwarding for RAW dependencies, generates 1-cycle stalls for load-use hazards, holds the pipeline during multi-cycle math (division / FPU), and flushes wrong-path instructions during branch mispredictions.

Q2: How does the Branch Target Buffer (BTB) achieve 0-cycle branch penalties?

It caches target addresses in the `IF` stage. When a branch PC hits in the BTB, the next fetch PC is redirected combinationally to the target in cycle N+1, allowing target instruction execution without inserting pipeline bubbles.

Q3: Why is the branch misprediction penalty exactly 2 cycles?

Branch conditions evaluate in the Execute stage (Stage 3). If mispredicted, the 2 younger speculative instructions already fetched into `IF/ID` (Stage 2) and `ID/EX` (Stage 1) must be flushed (3 − 1 = 2 cycles).

Q4: Why does the BTB use a 5-bit index from PC[6:2] instead of PC[4:0]?

RISC-V standard instructions are 4-byte aligned, meaning `PC[1:0]` is always `2'b00`. Indexing begins at bit 2 (`PC[6:2]`) to map directly to 32 entries ($2^5 = 32$) without wasted duplicate slots.

Q5: Why carry branch prediction metadata through the pipeline registers?

Carrying `pred_taken` and `pred_target` through `IF/ID` and `ID/EX` registers prevents race conditions, ensuring the `EX` stage evaluates the branch against the exact speculative decision that was made when that specific instruction was fetched.